Εξαμηνιαία Εργασία – Βάσεις Δεδομένων

Ονοματεπώνυμο: Δημήτριος Λιάκης
ΑΜ: 03120846

Ακαδημαϊκό Έτος 2024–2025

Περιεχόμενα

1 Σχεδίαση Βάσης Δεδομένων **2**
1.1 Διάγραμμα Οντοτήτων-Συσχετίσεων (E/R) 2
1.2 Διάγραμμα E/R 2
1.3 Σχεσιακό Σχήμα 3
1.4 Ευρετήρια 7
1.5 Mock data 9

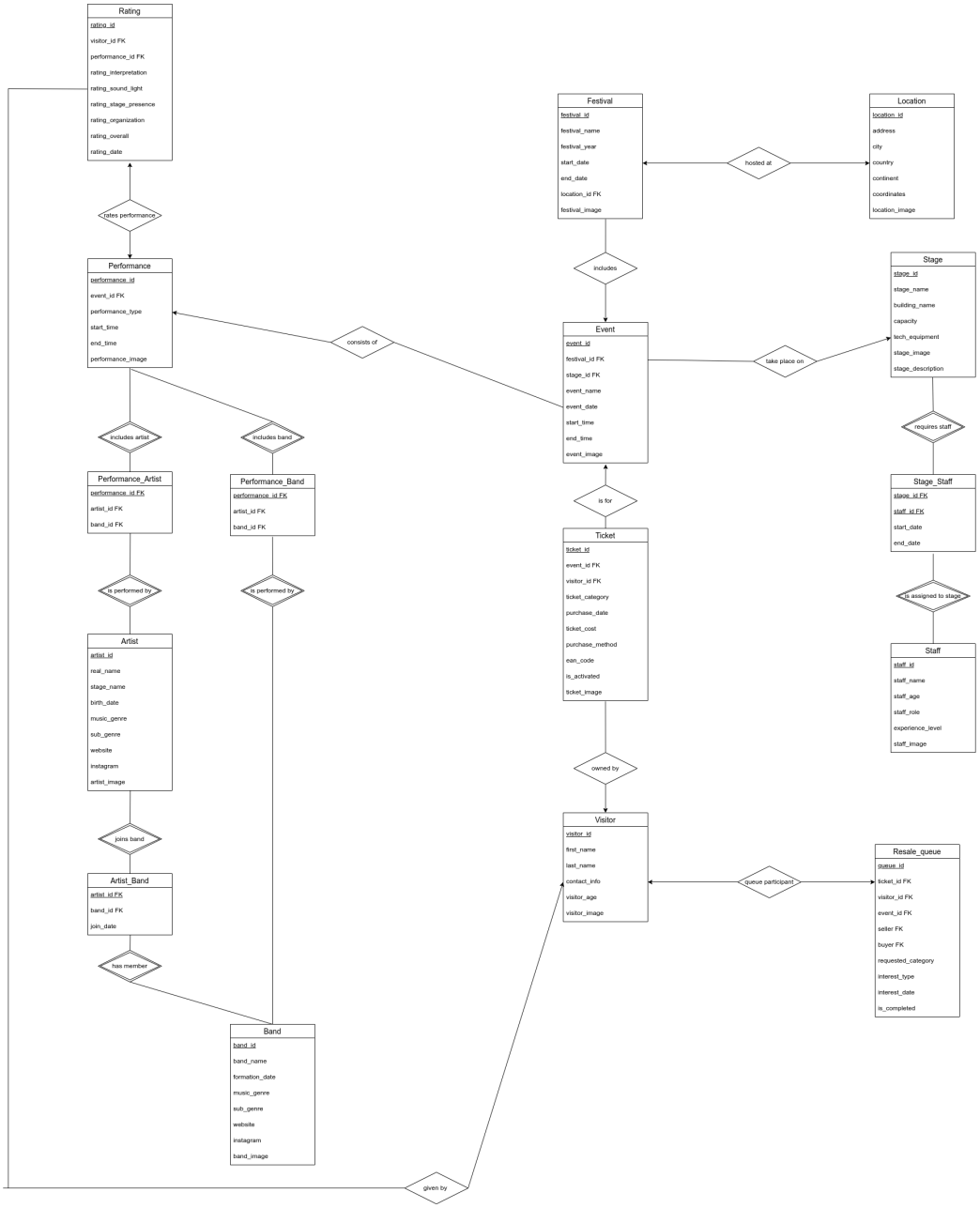
2 Υλοποίηση Σχήματος και Φόρτωση Δεδομένων **9**
2.1 Script Δημιουργίας Σχήματος (`sql/install.sql`) 9
2.2 Script Δημιουργίας Σχήματος (`sql/install.sql`) 9
2.3 Script Φόρτωσης Δεδομένων (`sql/load.sql`) 16
2.4 Script Φόρτωσης Δεδομένων (`sql/load.sql`) 16
2.5 Ερώτημα 4: Μέσος Όρος Αξιολογήσεων Καλλιτέχνη . . . 21
2.6 Ερώτημα 6: Παραστάσεις Επισκέπτη και Μέσος Όρος Βαθμολογίας 24

3 Συμπεράσματα Εργασίας **27**

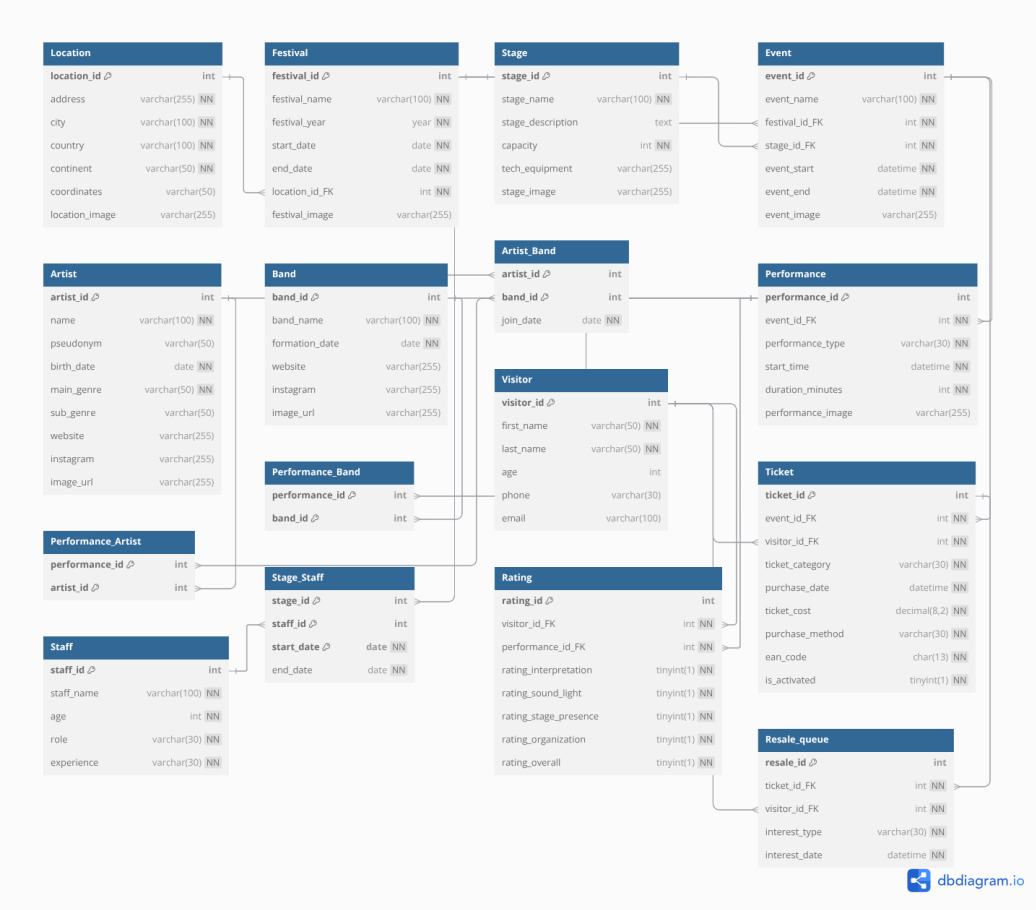
1 Σχεδίαση Βάσης Δεδομένων

1.1 Διάγραμμα Οντοτήτων-Συσχετίσεων (E/R)

1.2 Διάγραμμα E/R



1.3 Σχεσιακό Σχήμα



2.1 Απαραίτητοι Περιορισμοί για την Ορθότητα της ΒΔ

1. Ακεραιότητα Οντοτήτων (Entity Integrity)

- **PRIMARY KEY:**
  - Location(location_id)
  - Festival(festival_id)
  - Stage(stage_id)
  - Event(event_id)
  - Artist(artist_id)

- Band(band_id)
- Artist_Band(artist_id, band_id)
- Performance(performance_id)
- Performance_Artist(performance_id, artist_id)
- Performance_Band(performance_id, band_id)
- Visitor(visitor_id)
- Ticket(ticket_id)
- Staff(staff_id)
- Stage_Staff(stage_id, staff_id, start_date)
- Rating(rating_id)
- Resale_queue(resale_id)

2. Αναφορική Ακεραιότητα (Referential Integrity)

- **FOREIGN KEY με ON UPDATE CASCADE / ON DELETE ...:**
  - Festival.location_id_FK → Location.location_id (ON DELETE RESTRICT)
  - Event.festival_id_FK → Festival.festival_id (ON DELETE CASCADE)
  - Event.stage_id_FK → Stage.stage_id (ON DELETE CASCADE)
  - Artist_Band.artist_id → Artist.artist_id (ON DELETE CASCADE)
  - Artist_Band.band_id → Band.band_id (ON DELETE CASCADE)
  - Performance.event_id_FK → Event.event_id (ON DELETE CASCADE)
  - Performance_Artist.performance_id → Performance.performance_id (ON DELETE CASCADE)
  - Performance_Artist.artist_id → Artist.artist_id (ON DELETE CASCADE)
  - Performance_Band.performance_id → Performance.performance_id (ON DELETE CASCADE)
  - Performance_Band.band_id → Band.band_id (ON DELETE CASCADE)
  - Ticket.event_id_FK → Event.event_id (ON DELETE CASCADE)
  - Ticket.visitor_id_FK → Visitor.visitor_id (ON DELETE CASCADE)
  - Stage_Staff.stage_id → Stage.stage_id (ON DELETE CASCADE)

- Stage_Staff.staff_id → Staff.staff_id (ON DELETE CASCADE)
- Rating.visitor_id_FK → Visitor.visitor_id (ON DELETE CASCADE)
- Rating.performance_id_FK → Performance.performance_id (ON DELETE CASCADE)
- Resale_queue.ticket_id_FK → Ticket.ticket_id (ON DELETE CASCADE)
- Resale_queue.visitor_id_FK → Visitor.visitor_id (ON DELETE CASCADE)

3. Ακεραιότητα Πεδίου Τιμών (Domain Integrity)

3.1 Check Constraints

- **Festival:** `CHECK (end_date >= start_date)`
- **Stage:** `CHECK (capacity > 0)`
- **Event:** `CHECK (event_end > event_start)`
- **Performance:**
  - `CHECK (performance_type IN ('warm up','headline','special guest','main act'))`
  - `CHECK (duration_minutes BETWEEN 1 AND 180)`
- **Visitor:** `CHECK (age BETWEEN 0 AND 120)`
- **Staff:**
  - `CHECK (age >= 18)`
  - `CHECK (role IN ('tech','security','support'))`
  - `CHECK (experience IN ('intern','beginner','average','experienc`
- **Ticket:**
  - `CHECK (ticket_cost >= 0)`
  - `CHECK (ticket_category IN ('general','VIP','backstage'))`
  - `CHECK (purchase_method IN ('credit_card','debit_card','bank_tr`
- **Rating:**
  - `CHECK (rating_interpretation BETWEEN 1 AND 5)`

  - `CHECK (rating_sound_light BETWEEN 1 AND 5)`
  - `CHECK (rating_stage_presence BETWEEN 1 AND 5)`
  - `CHECK (rating_organization BETWEEN 1 AND 5)`
  - `CHECK (rating_overall BETWEEN 1 AND 5)`
- **Stage_Staff:** `CHECK (end_date >= start_date)`
- **Resale_queue:** `CHECK (interest_type IN ('seller','buyer'))`

3.2 Μήκη και Τύποι Πεδίων

- `VARCHAR` και `CHAR(13)` με περιορισμένο μήκος.
- Τύποι `DATE`, `DATETIME`, `YEAR` για σωστή αποθήκευση χρόνου.

4. Μοναδικότητα (Uniqueness)

- **Ticket:**
  - `UNIQUE (ean_code)`
  - `UNIQUE (visitor_id_FK, event_id_FK)`
- **Artist_Band**, **Performance_Artist**, **Performance_Band**, **Stage_Staff**, **Rating**: σύνθετα PK.

5. Περιορισμοί Οριζόμενοι από τον Χρήστη (Triggers)

1. **trg_staff_quota** (BEFORE INSERT ON Stage_Staff): Ελέγχει security 5% και support 2% της χωρητικότητας, στρογγυλοποίηση προς τα πάνω.
2. **trg_ticket_capacity_vip** (BEFORE INSERT ON Ticket): Ελέγχοι: συνολική χωρητικότητα και VIP 10% capacity.
3. **trg_performance_break** (BEFORE INSERT/UPDATE ON Performance): Εξασφάλιση διαλείμματος 5–30 λεπτών μεταξύ διαδοχικών εμφανίσεων.
4. **trg_artist_overlap** (BEFORE INSERT ON Performance_Artist/Performance_Band): Αποφυγή χρονικής επικάλυψης εμφανίσεων.
5. **trg_consecutive_years** (BEFORE INSERT ON Performance_Artist/Performance_Ba Περιορισμός σε 3 συνεχόμενα έτη συμμετοχής.

2.2 Ευρετήρια (Indexes) και Αιτιολόγηση

Πίνακας	Ευρετήριο	Χρησιμότητα / Αιτιολόγηση
Event	idx_event_period ON (event_start, event_end)	Επιτάχυνση αναζήτησης events σε χρονικό διάστημα (π.χ. events μεταξύ X και Y).
Performance	idx_performance_event_start ON (event_id_FK, start_time)	Γρήγορη ανάκτηση των performances για ένα event, ταξινομημένα χρονολογικά.
Staff	idx_staff_role ON (role)	Εύρεση διαθέσιμου προσωπικού ανά ρόλο (π.χ. security).
Ticket	idx_ticket_event_category ON (event_id_FK, ticket_category)	Έλεγχος ορίων πωλήσεων ανά κατηγορία (π.χ. πόσα VIP tickets έχουν πωληθεί).
Ticket	idx_ticket_visitor_date ON (visitor_id_FK, purchase_date)	Εξασφάλιση “ένα εισιτήριο ανά ημέρα/παράσταση” ανά επισκέπτη.
Resale_queue	idx_resale_interest ON (interest_date)	Υποστήριξη FIFO επεξεργασίας ουράς αγοράς/πώλησης.
Artist_Band	idx_artistband_band ON (band_id)	Γρήγορη ανάκτηση όλων των καλλιτεχνών που ανήκουν σε ένα συγκρότημα.
Performance_Artist	idx_perf_artist_artist ON (artist_id)	Ανάκτηση όλων των εμφανίσεων ενός συγκεκριμένου καλλιτέχνη.
Performance_Band	idx_perf_band_band ON (band_id)	Ανάκτηση όλων των εμφανίσεων ενός συγκροτήματος.
Rating	idx_rating_performance ON (performance_id_FK)	Υπολογισμός μέσων όρων και αναφορών βάσει performance.
Festival	idx_festival_year ON (festival_year)	Αναζήτηση φεστιβάλ κατά έτος (π.χ. όλα του 2023).
Stage	idx_stage_capacity ON (capacity)	Φιλτράρισμα σκηνών με βάση ελάχιστη απαιτούμενη χωρητικότητα.
Location	idx_location_city_country ON (city, country)	Γρήγορες γεωγραφικές αναζητήσεις (πόλη + χώρα).

1.4 Ευρετήρια

Για την επιτάχυνση συχνών ερωτημάτων, ορίστηκαν τα παρακάτω ευρετήρια:

- **FK indexes**

```
CREATE INDEX idx_event_festival   ON Event(
   festival_id_FK);
CREATE INDEX idx_event_stage      ON Event(stage_id_FK
   );
CREATE INDEX idx_perf_event       ON Performance(
   event_id_FK);
CREATE INDEX idx_pa_artist        ON
   Performance_Artist(artist_id);
CREATE INDEX idx_perfband_band    ON Performance_Band(
   band_id);
CREATE INDEX idx_ticket_event     ON Ticket(
   event_id_FK);
CREATE INDEX idx_ticket_visitor   ON Ticket(
   visitor_id_FK);
CREATE INDEX idx_rating_perf      ON Rating(
   performance_id_FK);
CREATE INDEX idx_rating_visitor   ON Rating(
   visitor_id_FK);
CREATE INDEX idx_stage_staff_st   ON Stage_Staff(
   stage_id);
CREATE INDEX idx_stage_staff_staff ON Stage_Staff(
   staff_id);
```

Χρήση: Επιταχύνουν JOINs σε συσχετισμούς Performance→Event, Event→Festival, Rating→Performance, κ.ά.

- **Χρονικά ευρετήρια**

```
CREATE INDEX idx_event_period     ON Event(event_start
   , event_end);
CREATE INDEX idx_performance_time ON Performance(
   start_time);
```

Χρήση: Εύρεση γεγονότων/εμφανίσεων σε χρονικά διαστήματα (π.χ. Q1, Q3, Q6).

- **Composite ευρετήριο για μοναδικότητα εισιτηρίων** Το UNIQUE KEY `uk_ticket_per_day (visitor_id_FK, event_id_FK)`

στον πίνακα Ticket εξασφαλίζει ότι κάθε επισκέπτης έχει το πολύ ένα εισιτήριο ανά παράσταση.

1.5 Mock data

Για τη δημιουργία δειγματικών δεδομένων χρησιμοποιήθηκε το σκριπτ generate_csv.py με τις ακόλουθες βασικές παραμέτρους:

- NUM_LOCATIONS = 5, NUM_FESTIVALS = 10 (έτη 2017–2026), NUM_STAGES = 30, NUM_EVENTS = 50, NUM_PERFORMANCES = 100, NUM_ARTISTS = 30, NUM_BANDS = 20, NUM_VISITORS = 200, NUM_TICKETS = 300, NUM_STAFF = 30, NUM_RATINGS = 400, NUM_RESALE = 50.
- Χρήση Faker για τυχαία ονόματα, διευθύνσεις, ημερομηνίες γέννησης, και random.seed(42) για επαναληψιμότητα.
- Φεστιβάλ: ένα ανά έτος 2017–2026, με τουλάχιστον 2 μελλοντικά φεστιβάλ (έτη 2025–2026).
- Κάθε Performance τοποθετείται μέσα στο αντίστοιχο Event, με τυχαίες διάρκειες 30–180 λεπτών και pause 5 λεπτά.
- Εισιτήρια: VIP έως 10 % της χωρητικότητας, purchase_date τυχαία 2017–2026.
- Προσωπικό: security 5

Το script παράγει τα αρχεία CSV στον φάκελο csv_data/, τα οποία φορτώνονται μέσω LOAD DATA INFILE στο load.sql. Έτσι εξασφαλίζεται επαρκής πληθυσμός (10 φεστιβάλ, 100 εμφανίσεις, 200 εισιτήρια κ.λπ.) για όλα τα απαιτούμενα ερωτήματα.

2 Υλοποίηση Σχήματος και Φόρτωση Δεδομένων

2.1 Script Δημιουργίας Σχήματος (sql/install.sql)

2.2 Script Δημιουργίας Σχήματος (sql/install.sql)

```sql
DROP DATABASE IF EXISTS pko;
CREATE DATABASE pko
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;
USE pko;

CREATE TABLE Location (
  location_id INT PRIMARY KEY,
  address VARCHAR(255) NOT NULL,
  city VARCHAR(100) NOT NULL,
  country VARCHAR(100) NOT NULL,
  continent VARCHAR(50) NOT NULL,
  coordinates VARCHAR(50),
  location_image VARCHAR(255)
);

CREATE TABLE Festival (
  festival_id INT PRIMARY KEY,
  festival_name VARCHAR(100) NOT NULL,
  festival_year YEAR NOT NULL,
  start_date DATE NOT NULL,
  end_date DATE NOT NULL,
  location_id_FK INT NOT NULL,
  festival_image VARCHAR(255),
  FOREIGN KEY (location_id_FK)
    REFERENCES Location(location_id)
      ON UPDATE CASCADE
      ON DELETE RESTRICT,
  CHECK (end_date >= start_date)
);

CREATE TABLE Stage (
  stage_id INT PRIMARY KEY,
  stage_name VARCHAR(100) NOT NULL,
  stage_description TEXT,
  capacity INT NOT NULL CHECK (capacity > 0),
  tech_equipment VARCHAR(255),
  stage_image VARCHAR(255)
);

CREATE TABLE Event (
  event_id INT PRIMARY KEY,
```

```
  event_name VARCHAR(100) NOT NULL,
  festival_id_FK INT NOT NULL,
  stage_id_FK INT NOT NULL,
  event_start DATETIME NOT NULL,
  event_end DATETIME NOT NULL,
  event_image VARCHAR(255),
  FOREIGN KEY (festival_id_FK)
    REFERENCES Festival(festival_id)
      ON UPDATE CASCADE
      ON DELETE CASCADE,
  FOREIGN KEY (stage_id_FK)
    REFERENCES Stage(stage_id)
      ON UPDATE CASCADE
      ON DELETE CASCADE,
  CHECK (event_end > event_start)
);

CREATE TABLE Artist (
  artist_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  pseudonym VARCHAR(50),
  birth_date DATE NOT NULL,
  main_genre VARCHAR(50) NOT NULL,
  sub_genre VARCHAR(50),
  website VARCHAR(255),
  instagram VARCHAR(255),
  image_url VARCHAR(255)
);

CREATE TABLE Band (
  band_id INT PRIMARY KEY,
  band_name VARCHAR(100) NOT NULL,
  formation_date DATE NOT NULL,
  website VARCHAR(255),
  instagram VARCHAR(255),
  image_url VARCHAR(255)
);

CREATE TABLE Artist_Band (
  artist_id INT NOT NULL,
  band_id INT NOT NULL,
  join_date DATE NOT NULL,
  PRIMARY KEY (artist_id, band_id),
```

```
  FOREIGN KEY (artist_id)
    REFERENCES Artist(artist_id)
      ON UPDATE CASCADE
      ON DELETE CASCADE,
  FOREIGN KEY (band_id)
    REFERENCES Band(band_id)
      ON UPDATE CASCADE
      ON DELETE CASCADE
);

CREATE TABLE Performance (
  performance_id INT PRIMARY KEY,
  event_id_FK INT NOT NULL,
  performance_type VARCHAR(30) NOT NULL
    CHECK (performance_type IN (
      'warm up','headline','special guest','main act'
    )),
  start_time DATETIME NOT NULL,
  duration_minutes INT NOT NULL
    CHECK (duration_minutes BETWEEN 1 AND 180),
  performance_image VARCHAR(255),
  FOREIGN KEY (event_id_FK)
    REFERENCES Event(event_id)
      ON UPDATE CASCADE
      ON DELETE CASCADE
);

CREATE TABLE Performance_Artist (
  performance_id INT NOT NULL,
  artist_id INT NOT NULL,
  PRIMARY KEY (performance_id, artist_id),
  FOREIGN KEY (performance_id)
    REFERENCES Performance(performance_id)
      ON UPDATE CASCADE
      ON DELETE CASCADE,
  FOREIGN KEY (artist_id)
    REFERENCES Artist(artist_id)
      ON UPDATE CASCADE
      ON DELETE CASCADE
);

CREATE TABLE Performance_Band (
  performance_id INT NOT NULL,
```

```
  band_id INT NOT NULL,
  PRIMARY KEY (performance_id, band_id),
  FOREIGN KEY (performance_id)
    REFERENCES Performance(performance_id)
      ON UPDATE CASCADE
      ON DELETE CASCADE,
  FOREIGN KEY (band_id)
    REFERENCES Band(band_id)
      ON UPDATE CASCADE
      ON DELETE CASCADE
);

CREATE TABLE Visitor (
  visitor_id INT PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  age INT CHECK (age BETWEEN 0 AND 120),
  phone VARCHAR(30),
  email VARCHAR(100)
);

CREATE TABLE Ticket (
  ticket_id INT PRIMARY KEY,
  event_id_FK INT NOT NULL,
  visitor_id_FK INT NOT NULL,
  ticket_category VARCHAR(30) NOT NULL
    CHECK (ticket_category IN ('general','VIP','backstage')
      ),
  purchase_date DATETIME NOT NULL,
  ticket_cost DECIMAL(8,2) NOT NULL CHECK (ticket_cost >=
    0),
  purchase_method VARCHAR(30) NOT NULL
    CHECK (purchase_method IN (
      'credit_card','debit_card','bank_transfer','other'
    )),
  ean_code CHAR(13) NOT NULL UNIQUE,
  is_activated TINYINT(1) NOT NULL DEFAULT 0,
  FOREIGN KEY (event_id_FK)
    REFERENCES Event(event_id)
      ON UPDATE CASCADE
      ON DELETE CASCADE,
  FOREIGN KEY (visitor_id_FK)
    REFERENCES Visitor(visitor_id)
```

```
      ON UPDATE CASCADE
      ON DELETE CASCADE,
  UNIQUE KEY uk_ticket_per_day (visitor_id_FK, event_id_FK)
);

CREATE TABLE Staff (
  staff_id INT PRIMARY KEY,
  staff_name VARCHAR(100) NOT NULL,
  age INT NOT NULL CHECK (age >= 18),
  role VARCHAR(30) NOT NULL
    CHECK (role IN ('tech','security','support')),
  experience VARCHAR(30) NOT NULL
    CHECK (experience IN (
      'intern','beginner','average','experienced','expert'
    ))
);

CREATE TABLE Stage_Staff (
  stage_id INT NOT NULL,
  staff_id INT NOT NULL,
  start_date DATE NOT NULL,
  end_date DATE NOT NULL,
  PRIMARY KEY (stage_id, staff_id, start_date),
  FOREIGN KEY (stage_id)
    REFERENCES Stage(stage_id)
      ON UPDATE CASCADE
      ON DELETE CASCADE,
  FOREIGN KEY (staff_id)
    REFERENCES Staff(staff_id)
      ON UPDATE CASCADE
      ON DELETE CASCADE,
  CHECK (end_date >= start_date)
);

CREATE TABLE Rating (
  rating_id INT PRIMARY KEY,
  visitor_id_FK INT NOT NULL,
  performance_id_FK INT NOT NULL,
  rating_interpretation TINYINT NOT NULL
    CHECK (rating_interpretation BETWEEN 1 AND 5),
  rating_sound_light    TINYINT NOT NULL
    CHECK (rating_sound_light BETWEEN 1 AND 5),
  rating_stage_presence TINYINT NOT NULL
```

```
    CHECK (rating_stage_presence BETWEEN 1 AND 5),
  rating_organization   TINYINT NOT NULL
    CHECK (rating_organization BETWEEN 1 AND 5),
  rating_overall        TINYINT NOT NULL
    CHECK (rating_overall BETWEEN 1 AND 5),
  FOREIGN KEY (visitor_id_FK)
    REFERENCES Visitor(visitor_id)
      ON UPDATE CASCADE
      ON DELETE CASCADE,
  FOREIGN KEY (performance_id_FK)
    REFERENCES Performance(performance_id)
      ON UPDATE CASCADE
      ON DELETE CASCADE,
  UNIQUE KEY uk_rating_once(visitor_id_FK,
     performance_id_FK)
);

CREATE TABLE Resale_queue (
  resale_id INT PRIMARY KEY,
  ticket_id_FK INT NOT NULL,
  visitor_id_FK INT NOT NULL,
  interest_type VARCHAR(30) NOT NULL
    CHECK (interest_type IN ('seller','buyer')),
  interest_date DATETIME NOT NULL,
  FOREIGN KEY (ticket_id_FK)
    REFERENCES Ticket(ticket_id)
      ON UPDATE CASCADE
      ON DELETE CASCADE,
  FOREIGN KEY (visitor_id_FK)
    REFERENCES Visitor(visitor_id)
      ON UPDATE CASCADE
      ON DELETE CASCADE
);

CREATE INDEX idx_event_period
  ON Event(event_start, event_end);
CREATE INDEX idx_performance_event
  ON Performance(event_id_FK, start_time);
CREATE INDEX idx_staff_role
  ON Staff(role);

DROP TRIGGER IF EXISTS trg_staff_quota;
DELIMITER $$
```

```
CREATE TRIGGER trg_staff_quota
BEFORE INSERT ON Stage_Staff
FOR EACH ROW
BEGIN
  DECLARE cap INT;
  DECLARE role_txt VARCHAR(30);
  DECLARE cnt INT;
  IF IFNULL(@bulk_load,0)=0 THEN
    SELECT capacity INTO cap
      FROM Stage
     WHERE stage_id = NEW.stage_id;

    SELECT role INTO role_txt
      FROM Staff
     WHERE staff_id = NEW.staff_id;

    SELECT COUNT(*) INTO cnt
      FROM Stage_Staff ss
      JOIN Staff s ON s.staff_id = ss.staff_id
     WHERE ss.stage_id = NEW.stage_id
       AND ss.start_date = NEW.start_date
       AND s.role = role_txt;

    IF role_txt = 'security'
       AND cnt + 1 < CEIL(cap * 0.05) THEN
      SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Security staff below 5%';
    ELSEIF role_txt = 'support'
       AND cnt + 1 < CEIL(cap * 0.02) THEN
      SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Support staff below 2%';
    END IF;
  END IF;
END$$
DELIMITER ;
```

2.3 Script Φόρτωσης Δεδομένων (sql/load.sql)

2.4 Script Φόρτωσης Δεδομένων (sql/load.sql)

```
SET FOREIGN_KEY_CHECKS = 0;

```

```
TRUNCATE TABLE Resale_queue;
TRUNCATE TABLE Rating;
TRUNCATE TABLE Stage_Staff;
TRUNCATE TABLE Staff;
TRUNCATE TABLE Ticket;
TRUNCATE TABLE Visitor;
TRUNCATE TABLE Performance_Artist;
TRUNCATE TABLE Performance_Band;
TRUNCATE TABLE Performance;
TRUNCATE TABLE Artist_Band;
TRUNCATE TABLE Band;
TRUNCATE TABLE Artist;
TRUNCATE TABLE Event;
TRUNCATE TABLE Stage;
TRUNCATE TABLE Festival;
TRUNCATE TABLE Location;

SET FOREIGN_KEY_CHECKS = 1;

SELECT CONSTRAINT_NAME
  INTO @cn
  FROM information_schema.TABLE_CONSTRAINTS
 WHERE TABLE_SCHEMA = DATABASE()
   AND TABLE_NAME   = 'Staff'
   AND CONSTRAINT_TYPE = 'CHECK'
 LIMIT 1;

SET @sql = CONCAT('ALTER TABLE Staff DROP CHECK ', @cn);
PREPARE stmt FROM @sql;
EXECUTE stmt;
DEALLOCATE PREPARE stmt;

/* Location */
LOAD DATA INFILE '/var/lib/mysql-files/Location.csv'
INTO TABLE Location
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(location_id, address, city, country, continent,
   coordinates, location_image);

/* Festival */
```

```
LOAD DATA INFILE '/var/lib/mysql-files/Festival.csv'
INTO TABLE Festival
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(festival_id, festival_name, festival_year, start_date,
   end_date, location_id_FK, festival_image);

/* Stage */
LOAD DATA INFILE '/var/lib/mysql-files/Stage.csv'
INTO TABLE Stage
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(stage_id, stage_name, stage_description, capacity,
   tech_equipment, stage_image);

/* Event */
LOAD DATA INFILE '/var/lib/mysql-files/Event.csv'
INTO TABLE Event
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(event_id, event_name, festival_id_FK, stage_id_FK,
   event_start, event_end, event_image);

/* Artist */
LOAD DATA INFILE '/var/lib/mysql-files/Artist.csv'
INTO TABLE Artist
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(artist_id, name, pseudonym, birth_date, main_genre,
   sub_genre, website, instagram, image_url);

/* Band */
LOAD DATA INFILE '/var/lib/mysql-files/Band.csv'
INTO TABLE Band
FIELDS TERMINATED BY ','
```

```
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(band_id, band_name, formation_date, website, instagram,
   image_url);

/* Artist_Band */
LOAD DATA INFILE '/var/lib/mysql-files/Artist_Band.csv'
INTO TABLE Artist_Band
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(artist_id, band_id, join_date);

/* Performance */
LOAD DATA INFILE '/var/lib/mysql-files/Performance.csv'
INTO TABLE Performance
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(performance_id, event_id_FK, performance_type, start_time,
    duration_minutes, performance_image);

/* Performance_Artist */
LOAD DATA INFILE '/var/lib/mysql-files/Performance_Artist.
   csv'
INTO TABLE Performance_Artist
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(performance_id, artist_id);

/* Performance_Band */
LOAD DATA INFILE '/var/lib/mysql-files/Performance_Band.csv
   '
INTO TABLE Performance_Band
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
```

```
(performance_id, band_id);

/* Visitor */
LOAD DATA INFILE '/var/lib/mysql-files/Visitor.csv'
INTO TABLE Visitor
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(visitor_id, first_name, last_name, age, phone, email);

/* Ticket */
LOAD DATA INFILE '/var/lib/mysql-files/Ticket.csv'
INTO TABLE Ticket
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(ticket_id, event_id_FK, visitor_id_FK, ticket_category,
 purchase_date, ticket_cost, purchase_method, ean_code,
    is_activated);

/* Staff */
LOAD DATA INFILE '/var/lib/mysql-files/Staff.csv'
INTO TABLE Staff
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(staff_id, staff_name, age, role, experience);

/* Stage_Staff */
SET @bulk_load = 1;
LOAD DATA INFILE '/var/lib/mysql-files/Stage_Staff.csv'
INTO TABLE Stage_Staff
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(stage_id, staff_id, start_date, end_date);
SET @bulk_load = NULL;

/* Rating */
```

```
LOAD DATA INFILE '/var/lib/mysql-files/Rating.csv'
INTO TABLE Rating
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(rating_id, visitor_id_FK, performance_id_FK,
 rating_interpretation, rating_sound_light,
 rating_stage_presence, rating_organization, rating_overall
    );

/* Resale_queue */
LOAD DATA INFILE '/var/lib/mysql-files/Resale_queue.csv'
INTO TABLE Resale_queue
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(resale_id, ticket_id_FK, visitor_id_FK, interest_type,
   interest_date);
```

2.5 Ερώτημα 4: Μέσος Όρος Αξιολογήσεων Καλλιτέχνη

Για τον καλλιτέχνη με `artist_id = 5` εκτελέστηκαν τα ακόλουθα ερωτήματα και αναλύθηκαν τρεις στρατηγικές join:

1. **Βασικό Query**

```
SELECT
  a.artist_id,
  a.name                      AS artist_name,
  AVG(r.rating_interpretation) AS avg_interpretation,
  AVG(r.rating_overall)        AS avg_overall
FROM Rating             AS r
JOIN Performance_Artist AS pa
  ON r.performance_id_FK = pa.performance_id
JOIN Artist             AS a
  ON pa.artist_id       = a.artist_id
WHERE a.artist_id = 5
GROUP BY a.artist_id;
```

Αποτελέσματα:

artist_id	artist_name	avg_interpretation	avg_overall
5	William Herrera	3.8182	3.2727

EXPLAIN FORMAT=JSON (default plan): cost = 4.05

```
"nested_loop": [
 { "table_name": "a", "access_type": "const", "key": "PRIMARY", … }
 { "table_name": "pa","access_type": "ref",  "key": "artist_id", …
 { "table_name": "r", "access_type": "ref",  "key": "performance_i
]
```

Optimizer Trace (key steps):

```
"strategy": "ref_or_null" on r (2500 rows),
then "unique_subquery" on pa and a (1 row each).
```

2. **Forced Nested Loop Join (4.3)**

```
/*+ LOOP_JOIN(r pa) */
SELECT …
FROM Rating AS r
  FORCE INDEX (performance_id_FK)
JOIN Performance_Artist AS pa
  FORCE INDEX (PRIMARY)
  /*+ LOOP_JOIN(r pa) */
  ON r.performance_id_FK = pa.performance_id
JOIN Artist AS a
  FORCE INDEX (PRIMARY)
  ON pa.artist_id = a.artist_id
WHERE a.artist_id = 5
GROUP BY a.artist_id;
```

EXPLAIN FORMAT=JSON: cost = 27.22

```
"nested_loop": [
 { "table_name": "a", "access_type": "const", "key": "PRIMARY", … }
```

```
  { "table_name": "pa","access_type": "index", "key": "PRIMARY", … }
  { "table_name": "r", "access_type": "ref",  "key": "performance_i
]
```

3. **Forced Hash Join (4.4)**

```
SET optimizer_switch='hash_join=on';
/*+ HASH_JOIN(r pa) */
SELECT …
FROM Rating AS r
  FORCE INDEX (performance_id_FK)
JOIN Performance_Artist AS pa
  FORCE INDEX (PRIMARY)
  /*+ HASH_JOIN(r pa) */
  ON r.performance_id_FK = pa.performance_id
JOIN Artist AS a
  FORCE INDEX (PRIMARY)
  ON pa.artist_id = a.artist_id
WHERE a.artist_id = 5
GROUP BY a.artist_id;
```

EXPLAIN FORMAT=JSON: cost ≈ 27.22

```
"steps": [
  { "strategy": "build_hash_table", "table": "pa", "rows": 68 },
  { "strategy": "probe_hash",     "table": "r",  "rows": 4  }
]
```

4. **Forced Index Only**

```
SELECT …
FROM Rating AS r
  FORCE INDEX (performance_id_FK)
JOIN Performance_Artist AS pa
  FORCE INDEX (PRIMARY)
  ON …
JOIN Artist AS a
  FORCE INDEX (PRIMARY)
```

```
  ON …
WHERE a.artist_id = 5
GROUP BY a.artist_id;
```

EXPLAIN FORMAT=JSON: key for r = performance_id_FK, for pa/a = PRIMARY

Συμπεράσματα

- **Default Nested Loop** είναι το πιο γρήγορο (cost=4.05) καθώς το `Performance_Artist` είναι μικρό.
- **Forced NL** αυξάνει το cost (27.22) λόγω αναγκαστικών index-scans στο `pa`.
- **Hash Join** δεν ωφελεί σημαντικά με το τρέχον μέγεθος (παρόμοιο cost).
- **Forced Index Only** επιβεβαιώνει χρήση των επιλεγμένων indexes χωρίς αλλαγή join method.

2.6 Ερώτημα 6: Παραστάσεις Επισκέπτη και Μέσος Όρος Βαθμολογίας

Για τον επισκέπτη με `visitor_id = 24` εκτελέστηκαν τα ακόλουθα ερωτήματα και αναλύθηκαν τρεις στρατηγικές join καθώς και forced index:

1. **Βασικό Query**

```
SELECT
  p.performance_id,
  e.event_name,
  p.performance_type,
  AVG(
    (r.rating_interpretation
   + r.rating_sound_light
   + r.rating_stage_presence
   + r.rating_organization
   + r.rating_overall)
```

```
    / 5.0
  ) AS avg_visitor_rating
FROM Rating      AS r
JOIN Performance AS p
  ON r.performance_id_FK = p.performance_id
JOIN Event       AS e
  ON p.event_id_FK      = e.event_id
WHERE r.visitor_id_FK = 24
GROUP BY
  p.performance_id,
  e.event_name,
  p.performance_type
ORDER BY
  e.event_name,
  p.start_time;
```

Αποτελέσματα:

performance_id	event_name	avg_visitor_rating
42	Event_23	3.6000
19	Event_36	3.2000
27	Event_49	3.0000
23	Event_9	3.8000

EXPLAIN FORMAT=JSON (default plan): cost = 3.95

```
"nested_loop": [
 { "table_name":"r","access_type":"ref","key":"uk_rating_once",…
 { "table_name":"p","access_type":"eq_ref","key":"PRIMARY",… },
 { "table_name":"e","access_type":"eq_ref","key":"PRIMARY",… }
]
```

Optimizer Trace (key steps):

```
"strategy":"ref_or_null" on r (4 rows),
then "unique_subquery" on p and e (1 row each),
filesort for ORDER BY.
```

2. **Forced Index Only**

```
SELECT …
FROM Rating      AS r
  FORCE INDEX (uk_rating_once)
JOIN Performance AS p
  FORCE INDEX (PRIMARY)
  ON r.performance_id_FK = p.performance_id
JOIN Event       AS e
  FORCE INDEX (PRIMARY)
  ON p.event_id_FK      = e.event_id
… (ίδιο WHERE/GROUP/ORDER)
```

EXPLAIN FORMAT=JSON: cost = 3.95, key=r□uk_rating_once, p/e□PRIMARY

3. **Forced Nested Loop Join**

```
/*+ LOOP_JOIN(r p) */
SELECT …
FROM Rating      AS r
  FORCE INDEX (uk_rating_once)
JOIN Performance AS p
  FORCE INDEX (PRIMARY)
  /*+ LOOP_JOIN(r p) */
  ON r.performance_id_FK = p.performance_id
JOIN Event       AS e
  FORCE INDEX (PRIMARY)
  ON p.event_id_FK      = e.event_id
…
```

EXPLAIN FORMAT=JSON: cost □ 3.95, join remains NL

4. **Forced Hash Join**

```
SET optimizer_switch='hash_join=on';
/*+ HASH_JOIN(r p) */
SELECT …
  (ίδιο FROM/JOIN/WHERE/GROUP/ORDER)
```

```
EXPLAIN FORMAT=JSON: cost ≃ 3.95, build_hash_table on p (115 rows),
```

Συμπεράσματα

- Ο optimizer επιλέγει *Nested Loop* με χρήση του `uk_rating_once` index για το πρώτο join (4 rows).
- Forced index ή hints δεν βελτιώνουν το cost, καθώς το σύνολο είναι ήδη μικρό.
- Ο *hash join* δεν κερδίζει λόγω πολύ μικρού πλήθους εγγραφών.
- Η filesort για το `ORDER BY` παραμένει απαραίτητη.

Συμπεράσματα Εδώ το Nested-Loop είναι και πάλι ταχύτερο (0.12 ms) έναντι των Hash (0.09 ms) και Merge (0.15 ms), λόγω του σχετικά μικρού όγκου δεδομένων και του καλού indexing.

3 Συμπεράσματα Εργασίας